

# Scaling Neural Network Verification with Tensor Parallelism and Fully Sharded Data Parallelism

Sergei Vorobyov<sup>1</sup>  and Eugene Ilyushin<sup>1,2</sup> 

<sup>1</sup> Lomonosov Moscow State University, Moscow, Russia

`sergei.vorobyov01@gmail.com`

<sup>2</sup> Central University, Moscow, Russia

`eugene.ilyushin@gmail.com`

**Abstract.** GPU memory limits the size of networks that formal verification can handle, and the usual assumption is that the weight matrices are what has to be split across GPUs. We tested this assumption in `auto-LiRPA` /  $\alpha, \beta$ -CROWN. We added two sharding methods from large-model training and measured where the memory actually goes. *Tensor Parallelism* (TP) splits the weights together with the CROWN  $A$ -matrices. On 2 GPUs it lowered peak memory by  $1.97\times$  and made propagation  $1.90\times$  faster; on 4 GPUs the factors were  $3.83\times$  and  $3.49\times$ . Its bounds are exact in the first sharded zone and get looser in later zones, which use IBP. *Fully Sharded Data Parallelism* (FSDP) splits only the weights. Its bounds match the single-GPU bounds bit for bit, but peak memory grew by 26–34% and propagation took up to  $3.2\times$  longer. Copying the weights from host memory one layer at a time needed less memory and less time than FSDP. A per-node memory trace explains this. At  $h=4096$  the  $A$ -matrices take 5.6 times more memory than the weights. In  $\beta$ -CROWN with Branch-and-Bound the peak grows linearly with the number of subdomains  $B$ , as  $21.3 + 0.372 B$  MB, and weights are 0.13% of it. We therefore expect more from splitting the subdomains across GPUs, and estimate 45.0% less memory per GPU at  $P=2$  and 67.5% at  $P=4$ .

**Keywords:** Neural network verification · Tensor parallelism · Fully sharded data parallelism · Bound propagation ·  $\alpha, \beta$ -CROWN · GPU memory.

## 1 Introduction

Neural networks are now used in vehicle control, avionics and medical decision support, where a wrong output can cause real harm. Testing is not enough there, since each test checks a single input and not the whole set of admissible inputs [8]. Formal verification proves that a property holds for *every* input in a region.

A network computes  $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ , and we want a property  $\psi(f(\mathbf{x}))$  to hold for every input  $\mathbf{x}$  in a set  $\mathcal{X}$ . Solvers search  $\mathcal{X}$  for an input that breaks  $\psi$ . They answer SAT with such a counterexample, or UNSAT when none exists and the property holds [12]. Most benchmarks check local robustness. Here every  $\mathbf{x}$  with  $\|\mathbf{x} - \mathbf{x}_0\|_p \leq \varepsilon$  around a sample  $\mathbf{x}_0$  of class  $c$  must also be classified as  $c$  [11].

Control tasks such as ACAS Xu use specifications over physical variables [6]. Networks and properties are exchanged in ONNX and the VNN-LIB format [5], which the VNN-COMP competition is built on [15,4,3]. The problem is NP-complete already for one hidden ReLU layer [25],  $\exists\mathbb{R}$ -complete for smooth activations [2] and undecidable for some recurrent models [17]. A *complete* verifier therefore needs Branch-and-Bound, which is exponential in the worst case. An *incomplete* one uses a polynomial relaxation instead.

### 1.1 The Memory Bottleneck and the Question We Ask

In practice most verifiers use *bound propagation*. The best known of them,  $\alpha, \beta$ -CROWN [27], won VNN-COMP every year from 2021 to 2024. It runs on a GPU and computes linear bounds on the network output. Everything it touches has to fit into the memory of one GPU. The weights of a single hidden layer with 262 144 neurons and 4096 inputs already take about 4 GB in float32.

Parameter sharding is the reflex borrowed from large-model training, and it presupposes that parameters dominate. This paper implements that reflex faithfully in a state-of-the-art verifier, measures it, and shows the presupposition to be wrong for verification: the memory sits in the tensors indexed by the specification and subdomain axes. That is a statement about where multi-GPU verification research needs to aim.

### Contributions.

1. Section 4 traces the memory of a CROWN pass node by node. For  $\beta$ -CROWN+BaB a sweep over the batch size fits  $M_{\text{peak}}(B) \approx 21.3 + 0.372 B$  MB. Weights take only 0.13% of this peak.
2. Our TP version of `auto_LirPA` uses custom `BoundLinearTP_Col/Row` operators (Sections 3.2, 5.1). With 2 GPUs peak memory is  $1.97\times$  lower and propagation  $1.90\times$  faster; with 4 GPUs the numbers are  $3.83\times$  and  $3.49\times$ . We also measure how much tightness IBP costs in each zone and check soundness for CROWN and  $\alpha$ -CROWN.
3. FSDP took about 100 lines on top of `auto_LirPA`, and the same code handles convolutions and a ViT. Its bounds match the single-GPU ones bit for bit. The problem is memory. At  $h=4096$  the peak is 27% higher than on one GPU, and even plain host-memory staging needs less (Sections 3.3, 5.4).
4. We give a target for future work. All batch-dependent terms grow linearly with the number of subdomains, so splitting them over  $P$  GPUs should cut memory per GPU by 45.0% at  $P=2$  and 67.5% at  $P=4$ , or fit  $P$  times more subdomains per round (Section 5.5).

Our implementation, a fork of `auto_LirPA` /  $\alpha, \beta$ -CROWN, with all experiment scripts and the collated measurement records, is publicly available.<sup>3</sup>

<sup>3</sup> <https://github.com/vorobyov01/damdid-2026-submission>

## 2 Background

### 2.1 Bound-Propagation Methods

*IBP* [28] propagates intervals layer by layer: for  $\mathbf{y} = W\mathbf{x} + \mathbf{b}$  it gives  $\mathbf{l}_y = W^+\mathbf{l}_x + W^-\mathbf{u}_x + \mathbf{b}$  and  $\mathbf{u}_y = W^+\mathbf{u}_x + W^-\mathbf{l}_x + \mathbf{b}$ , with  $W^\pm$  the positive and negative parts of  $W$ . It is cheap and loses tightness fast with depth. *CROWN* [28] instead relaxes each unstable ReLU with pre-activation  $x \in [l, u]$ ,  $l < 0 < u$ , by  $\lambda_l x + b_l \leq \text{ReLU}(x) \leq \frac{u}{u-l}(x-l)$  and composes the relaxations in a backward pass,

$$A = A_L W_L \cdots A_1 W_1, \quad \ell \leq A\mathbf{x} + \mathbf{b} \leq u, \quad (1)$$

with  $A_i = \text{diag}(\lambda_i)$ . Since  $A$  is linear in the original input, cross-variable dependencies survive across all layers, which is what makes CROWN much tighter than IBP.  $\alpha$ -CROWN [27] turns the fixed slopes into optimizable per-neuron parameters  $\alpha_i \in [0, 1]$  and tightens the bound by gradient ascent, without branching.

*Complete verification* [22] adds Branch-and-Bound: when the relaxation is too loose the problem is split, e.g. by fixing a ReLU to  $x \geq 0$  or  $x < 0$ , and bounds are recomputed per subdomain.  $\beta$ -CROWN encodes the split constraints as Lagrangian dual variables  $\beta$  inside the propagation, so thousands of subdomains are evaluated in one batched GPU pass with  $\alpha$  and  $\beta$  optimized jointly; BICCOS [18] strengthens the root bound with cutting planes from the search tree, while other complete verifiers replace the search itself, by continuous relaxation [23], clause learning [7], adaptive set representations [1] or mixed-integer encodings [21].

### 2.2 The auto\_LirPA Library

All our work builds on `auto_LirPA` [26], the library under  $\alpha, \beta$ -CROWN, which expresses bound propagation over a PyTorch computational graph and derives bounds for any intermediate node of an arbitrary architecture. It supports *batch BaB*: up to hundreds of thousands of open subproblems are bounded in a single pass. Any modification must therefore keep batched processing intact and stay compatible with the traced graph, which is what makes the sharding work non-trivial.

## 3 Sharding Bound Propagation

### 3.1 Primitives, and Why Not Pipeline Parallelism

*Tensor Parallelism* [19] splits the matrix products inside a layer. A column-parallel layer splits  $W \in \mathbb{R}^{k \times n}$  along the output dimension into  $W^{(r)} \in \mathbb{R}^{k \times n/P}$ , and each GPU computes part of the output from the full input. A row-parallel layer splits  $W$  along the input dimension, and the partial products are summed, so a Column-Row pair needs one `AllReduce`. *FSDP* [16] instead splits every parameter tensor across  $P$  GPUs. Before a layer runs, an `AllGather` rebuilds its

full weight, and the copy is dropped after the layer, so only one full weight is on the GPU at a time. We did not use *pipeline parallelism* [9]. The CROWN backward pass of Equation 1 goes through all layers, so a pipeline would have to send weight matrices between stages at every step, which amounts to TP with more communication. Both TP and FSDP run the whole pass on every GPU. TP splits the computation, while FSDP splits only the storage of the weights.

### 3.2 Tensor Parallelism

We added two operators, `BoundLinearTP_Col` and `BoundLinearTP_Row`, as subclasses of `BoundLinear`. Their `bound_backward` calls `AllReduce`. JIT tracing must not run the collective, since it then fails with `Tried to trace ProcessGroup`, so both operators also register a custom ONNX symbolic method. The function `tp_shard_bounded_module` takes any `BoundedModule`, including one loaded from ONNX, swaps pairs of consecutive `BoundLinear` nodes for their TP versions and shards the weights. A BFS pass then marks each Column–Row subgraph as a *sharded zone*, so that every rank handles its intermediate bounds in the same way.

Since TP shards both  $W$  and the  $A$ -matrices, the backward step  $A \leftarrow A \cdot A_i W_i$  is local inside a zone, and one `AllReduce` at the Row layer gives the exact result. The difficulty is the ReLU between the Column and the Row layer. To choose its slopes  $\lambda_i$ , CROWN needs bounds on the Column outputs. Computing them with CROWN would take another backward pass with communication at every step and would cost the memory we just saved. Each rank therefore bounds its own Column outputs with IBP. For the first zone this changes nothing, because the Column layer is one affine map of the input box and IBP gives the exact interval there. In later zones the input of the Column layer is already an interval, and IBP loses the correlation between its components. This is known as the *wrapping effect* [13].

*How fast tightness degrades.* Suppose IBP passes through  $k$  linear layers with ReLUs between them, and let  $w(\cdot)$  be the elementwise width of an interval. A linear layer maps widths by  $w(W\mathbf{x}) \leq |W|w(\mathbf{x})$  componentwise, and ReLU can only shrink a width,  $w(\text{ReLU}(x)) \leq w(x)$ . Together this gives

$$\|w_{\text{out}}\|_{\infty} \leq \left( \prod_{i=1}^k \|W_i\|_{\infty} \right) \|w_{\text{in}}\|_{\infty}. \quad (2)$$

The inequality bounds the widths that IBP produces. It does not give an exact growth rate for the gap between CROWN and IBP. It does show that this gap depends on the product of the layer norms and grows with every zone, which agrees with Table 2. For the three MNIST-FC models  $\|W\|_{\infty}$  is 19.6, 20.4 and 21.1. A two-variable example shows the effect. Take  $x_1, x_2 \in [-1, 1]$  and  $W = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ . CROWN finds  $y_1 + y_2 = 2x_1 \in [-2, 2]$  exactly, while IBP treats  $y_1, y_2 \in [-2, 2]$  as independent and returns  $[-4, 4]$ .

### 3.3 Fully Sharded Data Parallelism

`fsdp_shard_bounded_module` keeps on each GPU only its part of every `BoundParams` weight, cut along `dim=0`. A `BoundLinear` weight becomes  $W^{(r)} \in \mathbb{R}^{k/P \times n}$ , and a `BoundConv` weight becomes  $W^{(r)} \in \mathbb{R}^{C_{out}/P \times C_{in} \times k_H \times k_W}$ . Dimension 0 is the output axis in both cases, so an `AllGather` along it restores the exact matrix. If the first dimension is not divisible by  $P$ , we leave the tensor whole. At  $P=4$  this happens only for the classifier. The hooks `fsdp_gather_node` and `fsdp_free_node` sit in `backward_general` (CROWN) and `IBP_general` (IBP). They gather a weight right before its layer runs and free it right after. The whole change is about 100 lines of code. TP took about 500, because it needed new operators and a modified graph.

FSDP does the same floating-point operations in the same order as the single-GPU code. Only the place the weight tensor comes from is different, so IBP and CROWN bounds are *bitwise* identical. We checked this on eight configurations at  $P=2$  and  $P=4$ : MLPs of width 256 with depth 2, 4 and 6 under IBP and CROWN, and the ONNX models  $256 \times 2$  and  $256 \times 4$  under CROWN. This follows from the construction, so we use it as a regression test.

FSDP works with complete verification without changes to the algorithm. Each rank runs a normal  $\beta$ -CROWN+BaB iteration once its weights are gathered, and  $\beta$  is handled exactly as on one GPU. To shard `BoundConv` over output channels we changed three files. `fsdp_utils.py` now accepts `BoundConv` and avoids sharding a weight twice, `backward_bound.py` gathers and frees conv weights in the CROWN backward, and `interval_bound.py` frees them after IBP. Transformers needed three JIT fixes. During tracing `auto_LirPA` fixes reshape arguments at batch size 1, but BaB later uses larger batches. `BoundReshape` now inserts the actual batch size when  $\prod(\text{new\_shape}) = x.\text{numel}()$ , `BoundConcat` takes its tensor count from the trace, and `BoundConstantOfShape` updates its shape argument.

## 4 Where the Memory Actually Is

We read GPU memory from the `torch.cuda` counters and split it in two. The *resident* part  $R$  is allocated before `compute_bounds` starts. It holds the weights, the traced graph and the inputs. The *transient* part  $T = M_{\text{peak}} - R$  appears during the pass and holds  $A$ -matrices, intermediate bounds, gathered weights and collective buffers. Sharding can only reduce  $R$ .

For an MLP with  $n$  inputs and  $d$  hidden layers of width  $h$ , the weights take  $S_W = 4(nh + (d-1)h^2 + hm)$  bytes. The  $A$ -matrices are harder to count, because CROWN builds a new one for every layer whose intermediate bounds it needs. For a layer of width  $h$  the specification dimension is  $h$ , so with  $N$  inputs each  $A$ -tensor has shape  $[N, h, h]$  and a lower/upper pair takes  $8Nh^2$  bytes. With  $c$  pairs alive at the same time we get

$$M_{\text{peak}} \approx S_W + 8cNh^2 + S_0, \quad (3)$$

where  $S_0$  is the rest: graph metadata, inputs and intermediate bounds. To find  $c$  we logged the allocated memory after every node of a CROWN pass at  $h=4096$ ,

$d=4$ ,  $N=1$ . It went from 417 to 1844 MB, and most of the increments were exactly 128 MB, the size of one  $A$ -pair of shape  $[4096, 1, 4096]$ . When the pass finished, 18 such tensors were still alive. That gives  $c=9$  and 1152 MB of  $A$ -matrices next to 204.5 MB of weights. With more inputs or a wider output specification the  $A$ -matrices grow further, while the weights stay the same.

Equation 3 gives a ceiling for FSDP. Sharding turns  $S_W$  into  $S_W/P$  and does not touch the other terms, so no implementation can save more than

$$\eta_{\max} = \left(1 - \frac{1}{P}\right) \frac{S_W}{M_{\text{peak}}}. \quad (4)$$

For our MLPs  $\eta_{\max}$  is 5.3% at  $h=4096$ ,  $d=4$  and 5.2% at  $h=8192$ . It drops to 2.9% at  $d=8$  and rises to 7.9% at  $P=4$ . A real implementation also pays for the gathered weight and the NCCL buffers, and Section 5.4 shows that these costs are larger than the saving.

*In complete verification the axis is the subdomain count.* In  $\beta$ -CROWN+BaB the largest tensors are indexed by the number  $B$  of subdomains in the batch. This holds for the per-neuron  $\alpha$  slopes, whose shape is  $(\text{layers}) \times (\text{specs}) \times (B) \times (\text{neurons})$ , and also for the intermediate bounds and the  $A$ -matrices. The weights do not depend on  $B$ . We swept  $B$  from 128 to 4096 on `mnist-net_256x6` (Table 5) and fitted a line through the six points:

$$M_{\text{peak}}(B) \approx 21.3 + 0.372 B \quad \text{MB}, \quad (5)$$

Each batch-dependent term doubles when  $B$  doubles, while the weights stay at 2.03 MB, which is 0.13% of the peak at  $B=4096$ . One might expect the  $\alpha$  tensors to dominate, but at  $B=4096$  they take only 121 MB (7.9% of the peak). Intermediate bounds and  $A$ -matrices take 290 MB each (18.8%). We could not attribute the remaining  $\approx 54\%$  to a tensor class, but it also grows linearly with  $B$ , so the axis worth sharding is  $B$ .

## 5 Experiments

*Setup.* Unless stated otherwise, runs used  $4 \times$  NVIDIA RTX PRO 4000 Blackwell (24 GB, PCIe, no NVLink, driver 580.159.04), 48 vCPU, PyTorch 2.8.0+cu128, CUDA 12.8 and NCCL with `NCCL_P2P_DISABLE=1`, launched by `torchrun`, with seed 42, one discarded warm-up and three measured repeats. Each mode runs in its own process. Under FSDP the returned bound tensors keep an autograd graph alive, and in a shared process this memory would be counted in the peak of the next measurement. The ViT and CIFAR-100 checks of Section 5.6 ran on  $2 \times$  NVIDIA A40 (48 GB). We also measured the MLP configurations on the A40, and the peaks agree to four significant digits, since allocated memory depends only on tensor shapes.

**Table 1.** TP scaling. Peak memory and propagation time both scale close to linearly. Comm is the time inside `AllReduce`, including graph construction and waiting for the slower rank. The single-GPU peak at  $H=131\,072$  equals the TP=2 per-rank peak at  $H=262\,144$  measured on the A40, so the TP memory model is exact.

| Mode   | Peak (MB) | Reduction | Build (ms) | Propagation (ms) | Speed-up | Comm (ms) |
|--------|-----------|-----------|------------|------------------|----------|-----------|
| Single | 13 513.2  | 1.00×     | 4 279.5    | 546.6            | 1.00×    | 0.0       |
| TP=2   | 6 856.7   | 1.97×     | 22 679.8   | 287.7            | 1.90×    | 89.2      |
| TP=4   | 3 528.4   | 3.83×     | 11 392.1   | 156.5            | 3.49×    | 139.9     |

**Table 2.** TP=2 deviation from the single-GPU reference. A single zone fed by the input box agrees to machine precision; each further zone compounds the IBP substitution, consistently with Equation 2. Soundness holds throughout, and cross-rank deviation is zero everywhere.

| Model         | Zones | $\ W\ _\infty$ | $ \Delta lb $      | $ \Delta ub $      | Sound |
|---------------|-------|----------------|--------------------|--------------------|-------|
| PyTorch 256×2 | 1     | —              | $3 \times 10^{-8}$ | $3 \times 10^{-8}$ | ✓     |
| PyTorch 256×4 | 2     | —              | 2.51               | 2.49               | ✓     |
| ONNX 256×2    | 1     | 19.6           | $6 \times 10^{-8}$ | $6 \times 10^{-8}$ | ✓     |
| ONNX 256×4    | 2     | 20.4           | 5.64               | 3.98               | ✓     |
| ONNX 256×6    | 3     | 21.1           | 792                | 750                | ✓     |

### 5.1 Tensor Parallelism: Memory and Time

The model is  $f(\mathbf{x}) = W_2 \text{ReLU}(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$  with  $W_1$  Column-parallel and  $W_2$  Row-parallel following the Megatron-LM pattern [19];  $D=4096$ ,  $H=131\,072$ ,  $N=2048$ ,  $\varepsilon=0.01$ , CROWN;  $H$  is chosen so that the single-GPU baseline fits in 24 GB. The custom TP operators make JIT tracing much slower, but this cost is paid once per model, not once per query, so we time graph construction separately. Timed together, TP looks  $4.8\times$  slower than the baseline, although its propagation is faster.

Table 1 is the strongest positive result in this paper. The model has a single zone fed directly by the input box, so the IBP fallback of Section 3.2 is exact here. We measure tightness on smaller models of identical structure in Section 5.2, where one zone agrees with the single-GPU bounds to machine precision.

### 5.2 Tensor Parallelism: Soundness and Tightness

The reference is single-GPU CROWN. We run it and TP=2 on PyTorch MLPs and on the VNN-COMP 2022 MNIST-FC ONNX models [15] ( $\varepsilon=0.02$ ,  $L_\infty$ ). For each model we look at  $|\Delta lb|$  and  $|\Delta ub|$ , at one-sided soundness ( $lb_{\text{tp}} \leq lb_{\text{ref}}$ ,  $ub_{\text{tp}} \geq ub_{\text{ref}}$ ) and at whether the ranks agree.

The same holds for  $\alpha$ -CROWN. Compared with a single-GPU  $\alpha$ -CROWN run, TP=2 differs by at most  $|\Delta lb|_{\text{max}} = 4.47 \times 10^{-8}$  (mean  $2.14 \times 10^{-8}$ , relative  $3.64 \times 10^{-7}$ ), well within the tolerance of  $10^{-4}$ . We applied `tp_shard_bounded_module` to all three MNIST-FC ONNX models without manual changes. It walks

**Table 3.** FSDP against single-GPU, one mode per process, after the two memory fixes described below. Weight shards are  $1/P$  of the weights while peak memory rises;  $P=4$  is indistinguishable from  $P=2$  because the term it shards is not the term that dominates.

| Model         | Weight tensors (MB) |        |         | Peak (MB) |          |          |         |
|---------------|---------------------|--------|---------|-----------|----------|----------|---------|
|               | single              | FSDP=2 | offload | single    | FSDP=2   | FSDP=4   | offload |
| $h=1024, d=4$ | 15.1                | 7.6    | 0       | 149.6     | 200.6    | 200.6    | 184.6   |
| $h=4096, d=4$ | 204.5               | 102.3  | 0       | 1 944.9   | 2 476.0  | 2 475.9  | 2 271.6 |
| $h=8192, d=4$ | 792.9               | 396.5  | 0       | 7 593.9   | 9 551.8  | 9 551.5  | 8 759.0 |
| $h=4096, d=8$ | 460.5               | 230.3  | 0       | 7 934.0   | 10 181.1 | 10 180.9 | 9 720.7 |

**Table 4.** Bound-propagation time (ms). Parentheses: for FSDP, time inside `AllGather` (ms); for offloading, staged host-to-device volume (MB), whose copy time is contained in the total. The naive baseline is cheaper than sharding on every configuration.

| Model         | single | FSDP=2        | FSDP=4          | offload          |
|---------------|--------|---------------|-----------------|------------------|
| $h=1024, d=4$ | 23.6   | 30.1 (17.3)   | 54.6 (43.7)     | 21.5 (51 MB)     |
| $h=4096, d=4$ | 74.7   | 236.6 (166.7) | 286.7 (217.4)   | 103.3 (637 MB)   |
| $h=8192, d=4$ | 418.1  | 983.3 (583.5) | 1 128.1 (733.7) | 528.4 (2 427 MB) |
| $h=4096, d=8$ | 381.1  | 867.4 (510.8) | 1 003.0 (653.4) | 477.6 (2 350 MB) |

the graph in topological order, replaces the `BoundLinear` pairs, reruns a forward pass to refresh the shapes and marks the zones. All three models pass the checks above.

### 5.3 FSDP: Exactness, and Its Limits

On eight configurations (MLPs of width 256 at depths 2, 4, 6 under IBP and CROWN; ONNX  $256 \times 2$  and  $256 \times 4$  under CROWN) FSDP reproduces the single-GPU bounds with  $|\Delta lb| = |\Delta ub| = 0.00$ , an exact zero, at both  $P=2$  and  $P=4$ . There is one caveat. In the *optimized*  $\alpha/\beta$  path, a distributed run does not match a plain single-GPU run bit for bit. On `mnist-net_256x6` at  $\varepsilon=0.05$ , three of nine specification bounds differ, by at most  $6.10 \times 10^{-5}$  absolute and  $1.17 \times 10^{-7}$  relative. Sharding is not the cause. Our distributed launcher turns off the TorchScript fusers to avoid a strides assertion that fires only on non-primary GPUs. With the same fuser settings, a single GPU without any process group gives exactly the distributed bounds, while the default single-GPU run keeps its own values. FSDP itself is therefore exact, but a distributed  $\alpha, \beta$ -CROWN run can still differ from a single-GPU run for reasons unrelated to sharding. This matters when comparing multi-GPU verification results.

### 5.4 FSDP: Memory and Time

The weight shards are exactly  $1/P$  of the weights (Table 3), but peak memory rises by 26–34%, and the ceiling of Equation 4 explains why: at  $h=4096, d=4$

the whole weight term is 10.5% of the peak, so  $P=2$  could have saved at most 5.3% of it, less than the cost of staging a full weight. Going from  $P=2$  to  $P=4$  moves the peak by 0.1 MB out of 2476 while adding 50 ms of collectives. These numbers come from our FSDP code after two fixes. One removed a second full copy of the weight that every gather allocated, which lowered the peak by about 2%. The other made the gathered weights free after the pass. A shard, however, is still a view of the full weight tensor, so the full storage stays allocated. Releasing it would lower each FSDP peak by  $(1 - 1/P)S_W$ , which is less than the gap to the single-GPU peak in every row of Table 3.

Propagation is 2.3–3.2 $\times$  slower under FSDP=2 at  $h \geq 4096$  (1.3 $\times$  at  $h=1024$ , the smallest workload), and 57–70% of the wall-clock is spent inside `AllGather`: 166.7 of 236.6 ms at  $h=4096$ , 583.5 of 983.3 ms at  $h=8192$ . On a PCIe machine without NVLink this overhead cannot be avoided, because one full weight per layer crosses the bus at every intermediate-bound computation. A  $d=4$  pass at  $h=4096$  runs 19 collectives and moves 842 MB.

*The naive baseline is better.* A simpler alternative to sharding is to keep the weights in host memory and copy each one to the GPU when its layer runs. We implemented this baseline with the same hooks, using a host-to-device copy instead of the collective. No weights then stay on the GPU, which removes more than FSDP’s  $1/P$ , and the bounds match the single-GPU reference. The baseline wins on both memory and time. At  $h=4096$ ,  $d=4$  it needs 2271.6 MB against 2476.0 MB for FSDP and 103.3 ms against 236.6 ms, and the same holds at every size (Tables 3, 4). Activation checkpointing, the other simple alternative, does not apply here. The CROWN backward stores no forward activations that could be recomputed, and its  $A$ -matrices stay in memory because they are still needed.

That this simple baseline beats FSDP sums up our finding well. Both mechanisms copy a full weight into a pass whose peak comes from tensors they do not touch. At  $h=4096$  the copies that `auto_LirPA` keeps after the layer that used them take 531 MB, more than twice the 204.5 MB of weights that either mechanism removes.

## 5.5 Complete Verification: the Memory Is in the Subdomains

We use `mnist-net_256x6` [15] with  $\varepsilon=0.05$  and run exactly 10 BaB rounds. The flag `bab.force_synchronous` makes the single-GPU and FSDP runs do the same work. It fixes the batch and turns off batch enlargement, early stopping and pruning in iteration.

Two things follow. First, FSDP cannot help here and does not: at  $B=512$  the peak is 211.9 MB on one GPU and 214.9 MB per rank at both  $P=2$  and  $P=4$ , because there are 2.03 MB of weights to shard and the `AllGather` buffers cost more than that.

Second, Equation 5 makes the pay-off of sharding the subdomain axis calculable instead of speculative. Splitting a global batch  $B$  across  $P$  GPUs leaves each with  $B/P$  subdomains and hence  $21.3 + 0.372 B/P$  MB, i.e. 45.0% less

**Table 5.** Where BaB memory goes, by number of subdomains  $B$  (single GPU, MB). Every batch-scaled term doubles with  $B$ ; the weight term does not move.

| $B$  | $\alpha$ | $\beta$ | Interm. bounds | $A$ -matrices | Weights | Peak    |
|------|----------|---------|----------------|---------------|---------|---------|
| 128  | 3.79     | 0.01    | 9.06           | 9.08          | 2.03    | 68.73   |
| 256  | 7.58     | 0.02    | 18.13          | 18.16         | 2.03    | 116.24  |
| 512  | 15.16    | 0.05    | 36.26          | 36.33         | 2.03    | 211.93  |
| 1024 | 30.31    | 0.10    | 72.51          | 72.66         | 2.03    | 402.52  |
| 2048 | 60.62    | 0.20    | 145.03         | 145.32        | 2.03    | 780.72  |
| 4096 | 121.25   | 0.40    | 290.06         | 290.64        | 2.03    | 1543.61 |

memory per GPU at  $P=2$  and 67.5% less at  $P=4$  for  $B=512$  (49.3% and 73.9% at  $B=4096$ ). The per-GPU workload of the split is a single-GPU run at  $B/P$ , which Table 5 measures directly; what the projection leaves out is the cost of gathering the per-rank results. Since  $B$  is a tunable parameter, the law also reads the other way: at the same memory per GPU,  $P$  GPUs bound  $P$  times more subdomains per round. This throughput gain is the practical pay-off.

Our implementation of the split is partial. If the domains are scattered before `build_history_and_set_bounds`, each rank builds its own state and propagates its own chunk correctly. Domain parallelism does not compose with FSDP: once ranks hold different subdomains, their per-layer `AllGather` sequences diverge and NCCL deadlocks. We therefore disable weight sharding in this mode, which costs nothing here, the weights being 2.03 MB. The gather of the per-rank results still stalls, so we report the projection rather than a measurement.

## 5.6 Convolutions and Transformers: Correctness Checks

These runs show that the sharding code also works beyond MLPs. We treat them as correctness checks only, since memory changes by at most 3% in either direction. On the VNN-COMP’23 ViT `pgd_2_3_16` (2 blocks, 3 heads, 0.3 MB of weights) [4], after the three JIT fixes of Section 3.3, FSDP=2 completes 78 BaB domains over 10 rounds at 1020.1 MB per rank with  $|\Delta b| = 0.00$  at printed precision — to our knowledge the first Transformer run through BaB in  $\alpha, \beta$ -CROWN with sharded weights. On CIFAR-100 ResNets [3] with `BoundConv` sharding (batch 64, `max_iterations` 5), ResNet-med runs 130 domains over 5 rounds at 930 MB per rank against 960 MB on one GPU and exhausts its budget (*unknown*), while ResNet-large ( $\approx 95$  MB of weights) is proved **unsat** by  $\alpha$ -CROWN at initialization at 4641.5 MB per rank against 4610.6 MB on one GPU. That *unsat* is a correctness check, not a benefit: no BaB ran, and the FSDP peak is the higher of the two, its `AllGather` overhead exceeding what sharding 95 MB of weights returns when the  $\alpha$  tensors alone exceed 4 GB.

## 5.7 Reproducibility

All runs use seed 42. Each memory figure is the mean of three repeats after one warm-up. Its standard deviation was 0.00 MB in every configuration, so the

**Table 6.** Optimizer and search settings. All configuration files are in the repository; the fair-comparison configs additionally set `force_synchronous`, `auto_enlarge_batch_size`: `false`, `pruning_in_iteration`: `false` and `pgd_order`: `skip`.

| Setting                                                 | MNIST-FC 256×6 | ViT pgd_2_3_16 | CIFAR-100     | ResNet |
|---------------------------------------------------------|----------------|----------------|---------------|--------|
| $\alpha$ -CROWN lr / iters                              | 0.1 / 20       | 0.5 / 50       | 0.25 / 20     |        |
| $\beta$ -CROWN lr <sub><math>\alpha</math></sub>        | 0.1            | 0.05           | 0.1           |        |
| $\beta$ -CROWN lr <sub><math>\beta</math></sub> / iters | 0.03 / 20      | 0.05 / 10      | 0.2 / 10      |        |
| Batch                                                   | 128–4096       | 32             | 64            |        |
| BaB max_iterations                                      | 10             | 10             | 5             |        |
| Branching                                               | default        | reduceop: min  | kfsb, 7 cand. |        |
| $\varepsilon$                                           | 0.05           | benchmark      | 0.0039        |        |

tables have no error columns. Timings vary by less than 5% at  $h \geq 4096$ . At  $h=1024$  a pass takes only tens of milliseconds, and the jitter reaches 24% on one GPU and 43% under FSDP=4. The scripts for every table and the collated measurements (`experiments/analysis/RESULTS.md`) are in the repository.

## 6 Related Work

*Bound propagation.* CROWN [28] introduced the backward linear relaxation,  $\alpha$ -CROWN [27] and  $\beta$ -CROWN [22] added optimizable slopes and dual encoding of branching constraints, DeepPoly [20] works in a polyhedral abstract domain and OSIP [10] optimizes symbolic interval propagation.

*Parallel and multi-GPU verification.* Earlier parallel verifiers follow two lines, and neither deals with the memory of the parameters. The first splits the search space. Marabou’s Split-and-Conquer [24] splits the input domain across CPU workers with iterative deepening, and  $\beta$ -CROWN [22] batches subdomains on one GPU. The second moves a fixed abstract domain to the GPU, as GPUPoly [14] does for DeepPoly. Both scale the *number* of subproblems, while we looked at the memory of one. Our measurements show that this memory sits in the subproblem batch, so splitting domains and saving memory are the same problem.

*Sharding for training.* Megatron-LM [19] introduced intra-layer tensor parallelism, ZeRO [16] shards parameters, gradients and optimizer states, and GPipe [9] pipelines layer groups with micro-batches. To our knowledge neither had been applied to bound-propagation verification, which is what makes the negative result worth recording: a training step is dominated by parameters and optimizer state, a verification pass by tensors indexed by specifications and subdomains.

## 7 Conclusion

Of the two methods, only TP reduced memory. It splits the  $A$ -matrices together with the weights, so its memory and time scale almost linearly with the number

of GPUs. Its weak point is tightness, since only the first sharded zone keeps exact bounds and the later zones use IBP. FSDP kept the bounds bit-identical, but on our MLPs it raised peak memory, and copying the weights from host memory did better. This is because in our MLPs the weights are only 6–11% of the peak. Sharding them can save a few percent at most, which is less than the gathered copies cost.

For us the more useful result is the memory measurement itself. In incomplete verification the  $A$ -matrices take 5.6 times more memory than the weights at  $h=4096$ . In complete verification the peak is  $21.3 + 0.372 B$  MB for  $B$  subdomains, and weights are 0.13% of it. Multi-GPU verification should therefore split the subdomains. We estimate 45.0% and 67.5% less memory per GPU at  $P=2$  and  $P=4$ , or  $P$  times more subdomains per round at the same memory. In our code the scatter of subdomains already works on each rank, and the gather of per-rank results is still missing. A harder open problem is to shard the specification axis without the IBP fallback, so that the later sharded zones also get exact intermediate bounds.

**Acknowledgments.** The authors thank the `auto_LiRPA` /  $\alpha, \beta$ -CROWN team for the open-source codebase that made this work possible, and the anonymous reviewers, whose comments improved the memory analysis.

**Disclosure of Interests.** The authors have no competing interests to declare that are relevant to the content of this article.

## References

1. Bak, S.: nnenum: Verification of ReLU neural networks with optimized abstraction refinement (2021), [https://www.researchgate.net/publication/351683889\\_nnenum\\_Verification\\_of\\_ReLU\\_Neural\\_Networks\\_with\\_Optimized\\_Abstraction\\_Refinement](https://www.researchgate.net/publication/351683889_nnenum_Verification_of_ReLU_Neural_Networks_with_Optimized_Abstraction_Refinement)
2. Bertschinger, D., Hertrich, C., Jungeblut, P., Miltzow, T., Weber, S.: Training neural networks is  $\exists\mathbb{R}$ -complete. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 34 (2021), [https://proceedings.neurips.cc/paper\\_files/paper/2021/file/9813b270ed0288e7c0388f0fd4ec68f5-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2021/file/9813b270ed0288e7c0388f0fd4ec68f5-Paper.pdf)
3. Brix, C., Bak, S., Johnson, T.T., Wu, H.: The fifth international verification of neural networks competition (VNN-COMP 2024): Summary and results (2024), <https://arxiv.org/abs/2412.19985>
4. Brix, C., Bak, S., Liu, C., Johnson, T.T.: The fourth international verification of neural networks competition (VNN-COMP 2023): Summary and results (2023), <https://arxiv.org/abs/2312.16760>
5. Demarchi, S., Guidotti, D., Pulina, L., Tacchella, A.: VNN-LIB: The verification of neural networks library (2021), <https://www.vnnlib.org/>
6. Demarchi, S., Guidotti, D., Pulina, L., Tacchella, A.: Supporting standardization of neural networks verification with VNN-LIB and CoCoNET. In: Proceedings of the 6th Workshop on Formal Methods for ML-Enabled Autonomous Systems (FoMLAS 2023). Kalpa Publications in Computing, vol. 16, pp. 47–58 (2023), <https://easychair.org/publications/paper/Qgdn/open>

7. Duong, H., Nguyen, T., Dwyer, M.: NeuralSAT: A DPLL(T)-based framework for verifying deep neural networks (2023), <https://github.com/dynaroars/neuralsat>
8. Guidotti, D.: Enhancing neural networks through formal verification. In: Proceedings of the AI\*IA 2019 Doctoral Consortium (AI\*IA 2019 DC). CEUR Workshop Proceedings, vol. 2495, pp. 107–112. CEUR-WS.org (2019), <https://ceur-ws.org/Vol-2495/paper13.pdf>
9. Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, D., Chen, M.X., Lee, H., Ngiam, J., Le, Q.V., Wu, Y., Chen, Z.: GPipe: Efficient training of giant neural networks using pipeline parallelism. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 32 (2019), <https://arxiv.org/abs/1811.06965>
10. Kern, P., Kleine Büning, M., Sinz, C.: Optimized symbolic interval propagation for neural network verification (2022), <https://arxiv.org/abs/2212.08567>
11. Liu, J., Chen, L., Miné, A., Wang, J.: Input validation for neural networks via runtime local robustness verification. CoRR **abs/2002.03339** (2020), <https://arxiv.org/abs/2002.03339>
12. Marzari, L., Mastroeni, I., Farinelli, A.: Advancing neural network verification through hierarchical safety abstract interpretation (2025), <https://arxiv.org/abs/2505.05235>
13. Moore, R.E., Kearfott, R.B., Cloud, M.J.: Introduction to Interval Analysis. SIAM (2009)
14. Müller, C., Makarchuk, G., Singh, G., Püschel, M., Vechev, M.: Scaling polyhedral neural network verification on GPUs (2021), <https://files.sri.inf.ethz.ch/website/papers/mler2021neural.pdf>
15. Müller, M.N., Brix, C., Bak, S., Liu, C., Johnson, T.T.: The third international verification of neural networks competition (VNN-COMP 2022): Summary and results (2022), <https://arxiv.org/abs/2212.10376>
16. Rajbhandari, S., Rasley, J., Ruwase, O., He, Y.: ZeRO: Memory optimizations toward training trillion parameter models (2020), <https://arxiv.org/abs/1910.02054>
17. Ruan, W., Huang, X., Kwiakowska, M.: Reachability analysis of deep neural networks with provable guarantees. In: Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI) (2018), <https://www.ijcai.org/proceedings/2018/0368.pdf>
18. Shi, Z., Jin, Q., Kolter, J.Z., Jana, S., Hsieh, C.J., Zhang, H.: Scalable neural network verification with branch-and-bound inferred cutting planes. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 37 (2024), [https://proceedings.neurips.cc/paper\\_files/paper/2024/file/33d93e4dc57453e7667b20f62e7c0681-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2024/file/33d93e4dc57453e7667b20f62e7c0681-Paper-Conference.pdf)
19. Shoenybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B.: Megatron-LM: Training multi-billion parameter language models using model parallelism (2020), <https://arxiv.org/abs/1909.08053>
20. Singh, G., Gehr, T., Püschel, M., Vechev, M.: An abstract domain for certifying neural networks (2019), <https://ggndpsngh.github.io/files/DeepPoly.pdf>
21. Tjeng, V., Xiao, K., Tedrake, R.: Evaluating robustness of neural networks with mixed integer programming (2017), <https://arxiv.org/abs/1711.07356>
22. Wang, S., Zhang, H., Xu, K., Lin, X., Jana, S., Hsieh, C.J., Kolter, J.Z.: Beta-CROWN: Efficient bound propagation with per-neuron split constraints for neural network robustness verification. In: Advances in Neural Information Processing Systems (NeurIPS) (2021), <https://arxiv.org/abs/2103.06624>

23. Wu, H., Isac, O., Zeljić, A., Tagomori, T., Daggitt, M., Kokke, W., Refaeli, I., Amir, G., Julian, K., Bassan, S., Huang, P., Lahav, O., Wu, M., Zhang, M., Komendantskaya, E., Katz, G., Barrett, C.: Marabou 2.0: A versatile formal analyzer of neural networks (2024), <https://arxiv.org/html/2401.14461v1>
24. Wu, H., Ozdemir, A., Zeljić, A., Julian, K., Irfan, A., Gopinath, D., Fouladi, S., Katz, G., Păsăreanu, C., Barrett, C.: Parallelization techniques for verifying neural networks. In: Formal Methods in Computer Aided Design (FMCAD). pp. 128–137 (2020)
25. Wurm, A.: Robustness verification in neural networks (2024), <https://arxiv.org/abs/2403.13441>
26. Xu, K., Shi, Z., Zhang, H., Wang, Y., Chang, K.W., Huang, M., Kailkhura, B., Lin, X., Hsieh, C.J.: Automatic perturbation analysis for scalable certified robustness and beyond (2020), [https://github.com/Verified-Intelligence/auto\\_LiRPA](https://github.com/Verified-Intelligence/auto_LiRPA)
27. Zhang, H., Wang, S., Xu, K., Li, L., Li, B., Jana, S., Hsieh, C.J., Kolter, J.Z.: alpha-beta-CROWN: A complete and efficient bound propagation based neural network verifier (VNN-COMP 2021–2024) (2023), [https://github.com/Verified-Intelligence/alpha-beta-CROWN\\_vnncomp23](https://github.com/Verified-Intelligence/alpha-beta-CROWN_vnncomp23)
28. Zhang, H., Weng, T.W., Chen, P.Y., Hsieh, C.J., Daniel, L.: Efficient neural network robustness certification with general activation functions (2018), <https://arxiv.org/abs/1811.00866>